End-to-End Fraud Detection System — Results Showcase

1. Introduction

1.1 Project Overview.

This report presents the analytical results generated by the fraud detection pipeline. The project was implemented in both a local development environment and a cloud-native architecture based on Azure Data Lake Storage (ADLS), Azure - Databricks, and Apache Airflow. Since both implementations execute the same data processing, feature engineering, model training, and evaluation logic, the analytical findings presented in this document are representative of the complete solution regardless of the execution environment.

1.2 Dataset.

Table 1 summarizes the main characteristics of the Credit Card Fraud Detection Dataset used throughout this project. The dataset contains 284,807 transactions, of which only 492 correspond to fraudulent activities, representing approximately 0.1727% of the observations. This results in an extreme class imbalance of roughly 578 non-fraudulent transactions for every fraudulent transaction, making fraud detection a highly challenging machine learning problem and emphasizing the importance of using evaluation metrics specifically designed for imbalanced datasets.

Attribute	Value
Dataset Name	Credit Card Fraud Detection Dataset
Source	Kaggle
Dataset Identifier	mlg-ulb/creditcardfraud
Total Transactions	284,807
Features	30 predictor variables
Target Variable	Class
Fraud Transactions	492
Non-Fraud Transactions	284,315
Fraud Rate	0.1727%
Class Imbalance Ratio	577.88 : 1

Table 1. *Credit Card Fraud Detection Dataset Summary.*

1.3 Models Evaluated.

Four machine learning algorithms were evaluated throughout the project to assess their ability to detect fraudulent transactions under severe class imbalance conditions. Table 2 shows the models evaluated.

Model	Description
Logistic Regression	Interpretable linear baseline model.
Random Forest	Ensemble tree-based classifier using bagging.
XGBoost	Gradient boosting framework optimized for predictive performance.
LightGBM	Gradient boosting framework optimized for computational efficiency.

Table 2. Machine Learning Models Evaluated.

1.4 Final Selected Model.

Although XGBoost achieved the strongest predictive performance according to PR-AUC and ROC-AUC metrics, **Random Forest was selected** as the production candidate for the final architecture. The decision was based on a broader operational evaluation that considered precision, false positive control, model stability, interpretability, and integration with the cloud-based PySpark/Databricks ecosystem.

Random Forest demonstrated a strong balance between fraud detection capability and operational reliability, generating substantially fewer false positives while maintaining competitive predictive performance. This behavior makes it more suitable for real-world fraud monitoring scenarios, where excessive false alerts can increase investigation costs and negatively impact customer experience.

Category	Selected Model
Best Predictive Performance	XGBoost
Production Candidate	Random Forest

Table 3. Final Model Selection Summary.

2. Visual Results

2.1 Class Imbalance Visualization.

Figure 1 illustrates the class distribution of the Credit Card Fraud Detection Dataset. The dataset contains 284,315 legitimate transactions and only 492 fraudulent

transactions, meaning that fraudulent observations represent approximately 0.17% of all records. This corresponds to an imbalance ratio of roughly 578 non-fraudulent transactions for every fraudulent transaction.

The visualization highlights the extreme rarity of fraud events, a characteristic commonly observed in real-world financial fraud detection problems. Such a severe imbalance increases the difficulty of the classification task, since a model can achieve very high overall accuracy while still failing to identify fraudulent transactions effectively. Consequently, model evaluation throughout this project focused primarily on Precision, Recall, F1-score, and PR-AUC rather than relying exclusively on Accuracy or ROC-AUC.

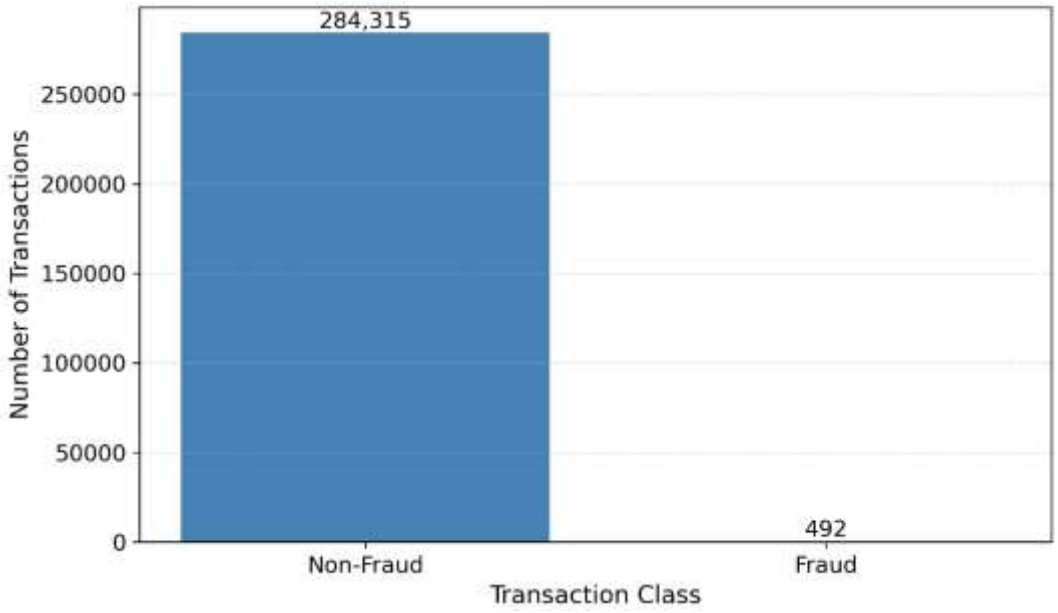

Figure 1. Class Imbalance in Fraud Detection Dataset.

2.2 Classification threshold analysis.

Figure 2 presents the threshold analysis for the four evaluated models. As expected, increasing the classification threshold generally improves Precision while reducing Recall, illustrating the inherent tradeoff between fraud detection sensitivity and false positive control.

Logistic Regression exhibits the highest Recall across most threshold values, maintaining fraud detection rates above 80% even at relatively strict thresholds. However, this behavior is accompanied by extremely low Precision, indicating that a large number of legitimate transactions would be incorrectly flagged as fraudulent.

In contrast, Random Forest and XGBoost achieve Precision values consistently above 94%, demonstrating a much stronger ability to generate reliable fraud alerts while maintaining competitive Recall levels. The F1-score comparison further highlights the differences between models. XGBoost achieves the highest overall F1-score at lower thresholds, reflecting its superior predictive performance, while Random Forest maintains a similarly strong and stable balance between Precision and Recall across a wide threshold range. LightGBM shows considerably lower performance, with limited sensitivity and substantially lower F1-scores throughout the analysis.

Overall, the threshold analysis confirms that Random Forest and XGBoost provide the most favorable tradeoff between fraud detection capability and false positive control. Logistic Regression prioritizes sensitivity at the expense of precision, whereas LightGBM demonstrates weaker discrimination ability under the evaluated conditions.

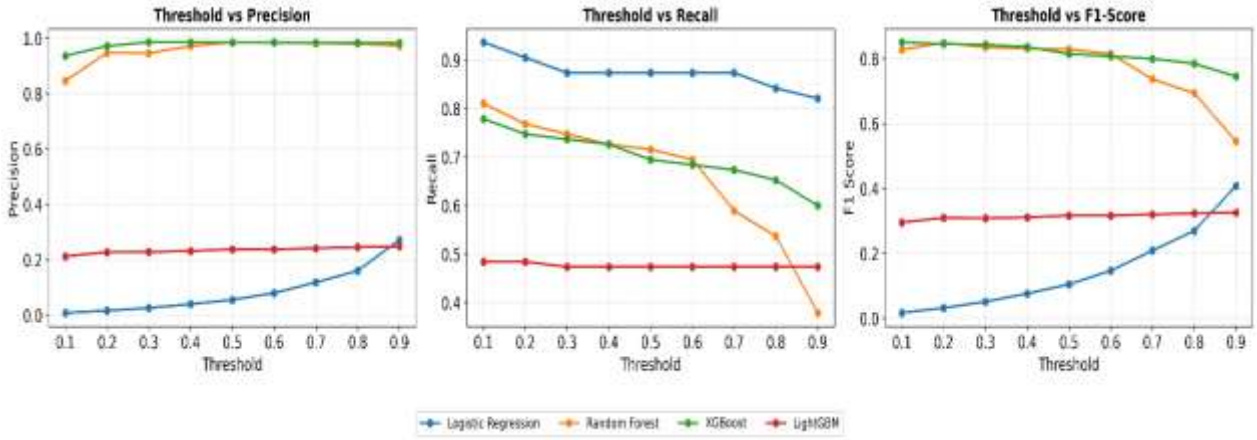

Figure 2. Threshold Analysis Across Fraud Detection Models.

2.3 Precision-Recall Curves.

Figure 3 presents the Precision-Recall (PR) curves for the four evaluated fraud detection models. Given the extreme class imbalance of the dataset, where fraudulent transactions represent less than 0.2% of all observations, PR curves provide a more informative assessment of model performance than accuracy alone.

XGBoost achieved the highest overall performance with a PR-AUC of approximately 0.828, closely followed by Random Forest with a PR-AUC of 0.825. Both models maintained high precision across a wide range of recall values, demonstrating a strong ability to identify fraudulent transactions while limiting false positive alerts. Logistic Regression obtained a lower PR-AUC of 0.710, indicating weaker

discrimination capability despite its high sensitivity observed during threshold analysis. LightGBM showed the poorest performance, with a PR-AUC of 0.317, reflecting limited effectiveness in separating fraudulent and legitimate transactions.

Overall, the PR curve analysis confirms that tree-based ensemble methods substantially outperform Logistic Regression in this fraud detection task. XGBoost delivered the strongest predictive performance, while Random Forest achieved nearly equivalent results and maintained the operational stability that motivated its selection as the production model for the final pipeline architecture.

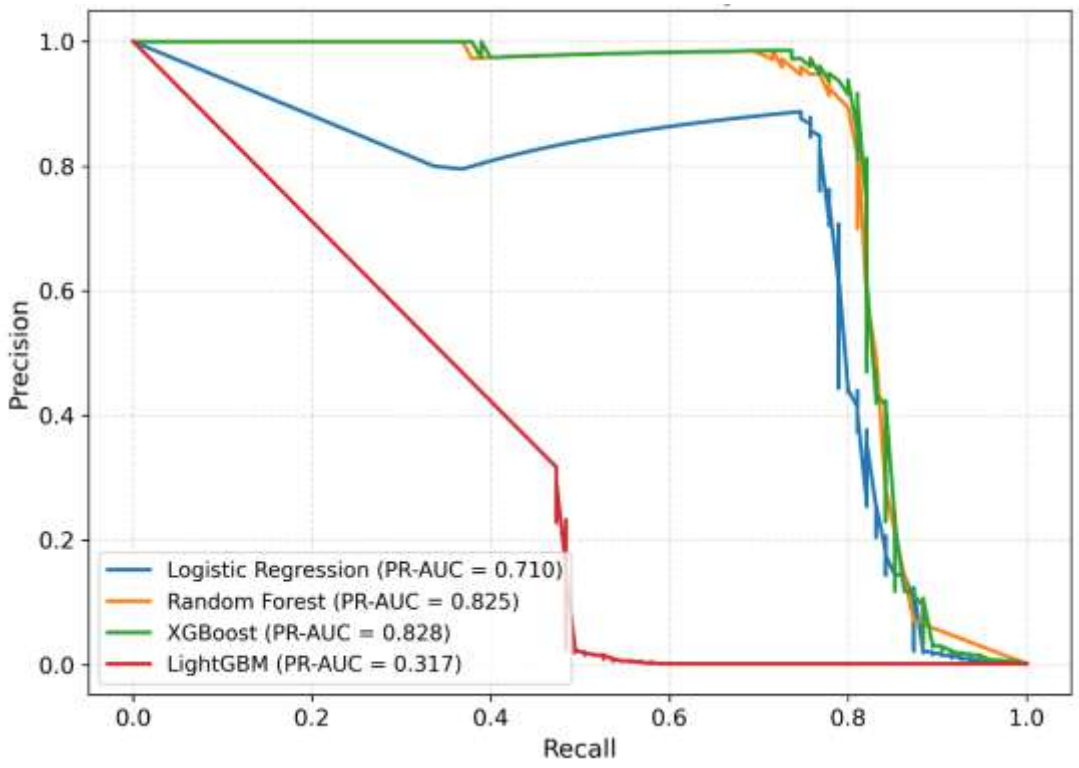

***Figure 3. Precision-Recall* Curve Comparison Across Fraud Detection Models.**

2.4 ROC curves.

Figure 4 presents the ROC curves for the four evaluated fraud detection models. The ROC curve evaluates the ability of a model to distinguish between fraudulent and legitimate transactions across all possible classification thresholds. Higher curves and larger ROC-AUC values indicate better overall discrimination performance.

XGBoost achieved the highest ROC-AUC (0.978), demonstrating the strongest overall classification capability among the evaluated models. Logistic Regression also produced excellent discrimination performance, reaching a ROC-AUC of 0.965, while Random Forest achieved a ROC-AUC of 0.935. In contrast, LightGBM showed substantially weaker performance, with a ROC-AUC of 0.665, remaining considerably closer to the random classifier reference line.

The ROC analysis confirms that Logistic Regression, Random Forest, and XGBoost all possess strong predictive power for separating fraudulent from legitimate transactions. However, as previously observed during threshold analysis and Precision-Recall evaluation, ROC-AUC alone does not fully capture operational performance in highly imbalanced datasets. For this reason, model selection was primarily guided by Precision, Recall, F1-score, PR-AUC, and false positive control. Under these criteria, **Random Forest was selected** as the production model despite XGBoost achieving the highest ROC-AUC and overall predictive performance.

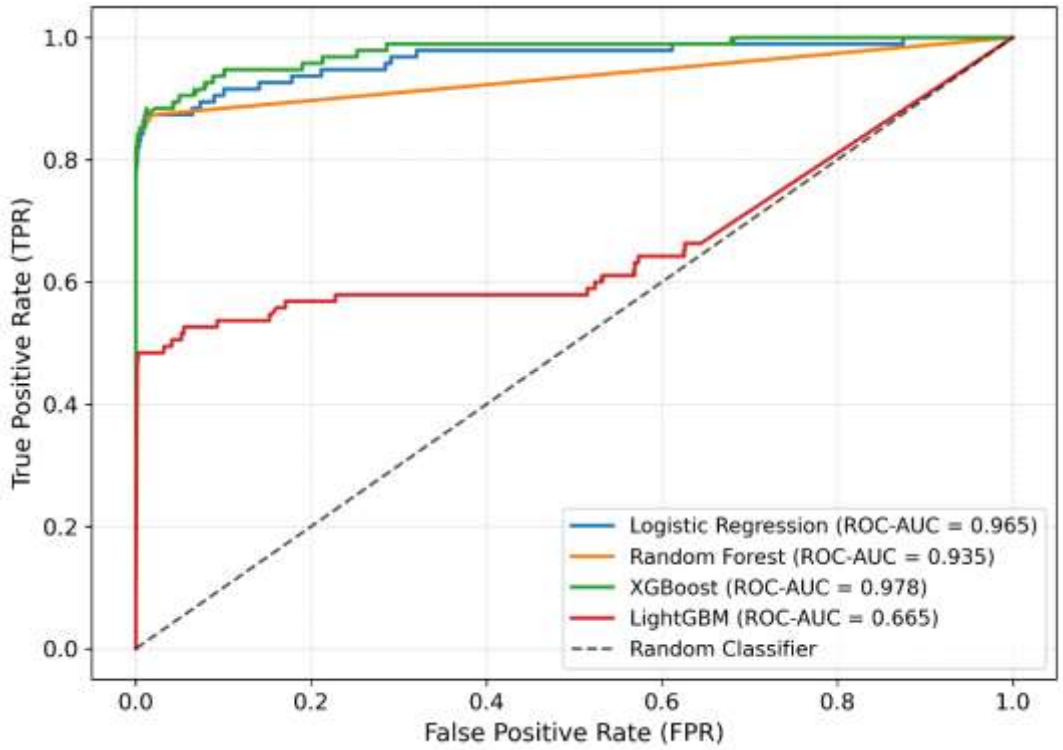

Figure 4. ROC Curve Comparison Across Fraud Detection Models.

2.5 Confusion Matrix

Table 4 summarizes the confusion matrix results obtained for the four evaluated fraud detection models. The results highlight the operational trade-offs associated

with each approach. Logistic Regression achieved the highest fraud detection rate, correctly identifying 83 fraudulent transactions while missing only 12. However, this performance came at the cost of 1,407 false positive alerts, which would generate a substantial operational burden in a real-world fraud investigation environment.

Random Forest and XGBoost produced a markedly different behavior. Both models generated only one false positive transaction while maintaining strong fraud detection capabilities, correctly identifying 68 and 66 fraudulent transactions, respectively. This demonstrates a significantly better balance between fraud detection and alert reliability. LightGBM exhibited the weakest overall performance, producing a higher number of missed fraud cases and more false positives than the other tree-based models.

From an operational perspective, these results explain the final model selection. While Logistic Regression maximized fraud detection, the extremely high number of false positives would require unnecessary investigations and could negatively affect customer experience. **Random Forest** achieved the most balanced outcome by maintaining strong fraud detection performance while virtually eliminating false positive alerts, making it the most suitable candidate for deployment within the production pipeline.

Model	TN	FP	FN	TP
Logistic Regression	55244	1407	12	83
Random Forest	56650	1	27	68
XGBoost	56650	1	29	66
LightGBM	56507	144	50	45

Table 4. Confusion Matrix Results Across Fraud Detection Models.

3. Pipeline Engineering

The results presented throughout this report were generated using an end-to-end fraud detection pipeline implemented in both local and cloud environments. Although the underlying execution infrastructure differs, both implementations follow the same analytical workflow, data transformations, feature engineering procedures, model training strategy, and evaluation methodology. Consequently, the predictive results, model comparisons, and performance metrics remain consistent across both execution modes.

To ensure reproducibility and automation, both architectures were orchestrated using Apache Airflow. The following diagrams summarize the local and cloud implementations used throughout the project.

3.1 Pipeline Architecture Diagram: local orquestation

Figure 5 illustrates the local implementation of the fraud detection pipeline. In this architecture, data extraction, preprocessing, feature engineering, model training, and model evaluation are executed locally through modular Python components coordinated by Apache Airflow. This implementation was used during the initial development and validation stages of the project and produces the same analytical outputs presented in the previous sections.

Figure 6 demonstrates the successful execution of the local orchestration workflow in Apache Airflow. The pipeline stages were coordinated through BashOperator tasks, enabling the automated execution of the modular Python components that compose the end-to-end fraud detection workflow.

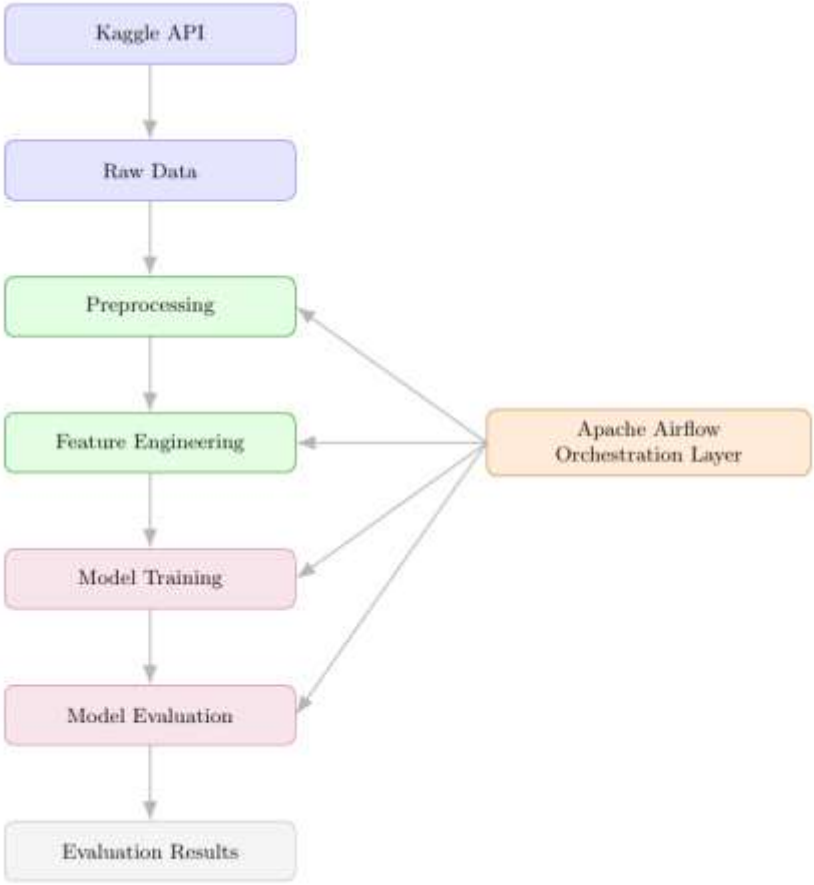

Figure 5. Local Orchestration Workflow for Fraud Detection.

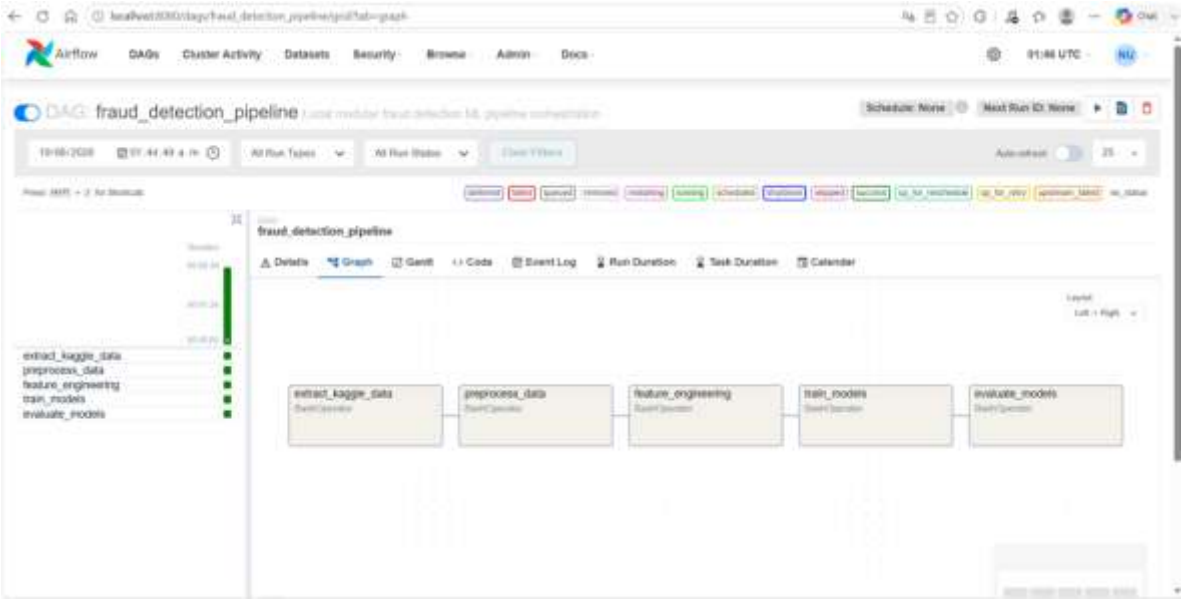

Figure 6. Local Pipeline Orchestration with Apache Airflow.

3.2 Pipeline Architecture Diagram: cloud orquestation

Figure 7 illustrates the cloud implementation of the fraud detection pipeline. In this architecture, the project source code was maintained in a GitHub repository and integrated with Azure Databricks, where the fraud detection workflow was configured and executed as a Databricks Job. Apache Airflow was used as the orchestration layer, responsible for triggering and monitoring the Databricks Job as part of the automated pipeline execution process.

Data storage and processing were performed using Azure services, including Azure Data Lake Storage (ADLS) and Azure Databricks. The cloud implementation preserves the same analytical workflow, feature engineering logic, model training strategy, and evaluation framework used in the local environment, ensuring that the results presented in this report remain fully reproducible across both architectures.

Figure 8 presents the Azure Databricks Job used to execute the cloud-based fraud detection workflow. The figure shows the sequence of tasks configured within the Databricks environment, including data extraction, preprocessing, feature engineering, model training, and model evaluation. Figure 9 demonstrates the successful orchestration of the cloud pipeline through Apache Airflow, where a DatabricksRunNowOperator task was used to automatically trigger and monitor the execution of the Databricks Job as part of the end-to-end workflow.

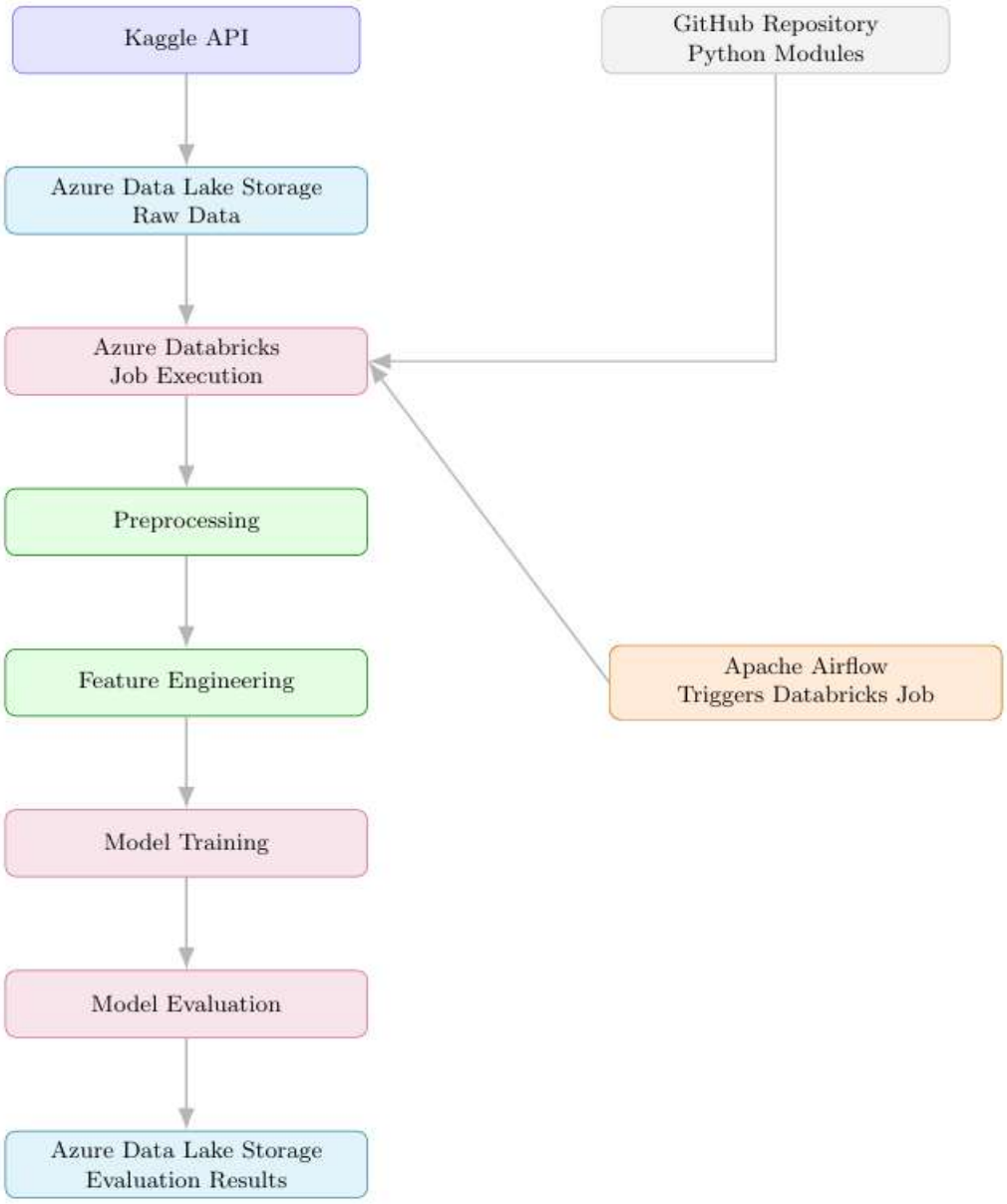

Figure 7. End-to-End Cloud Fraud Detection Pipeline.

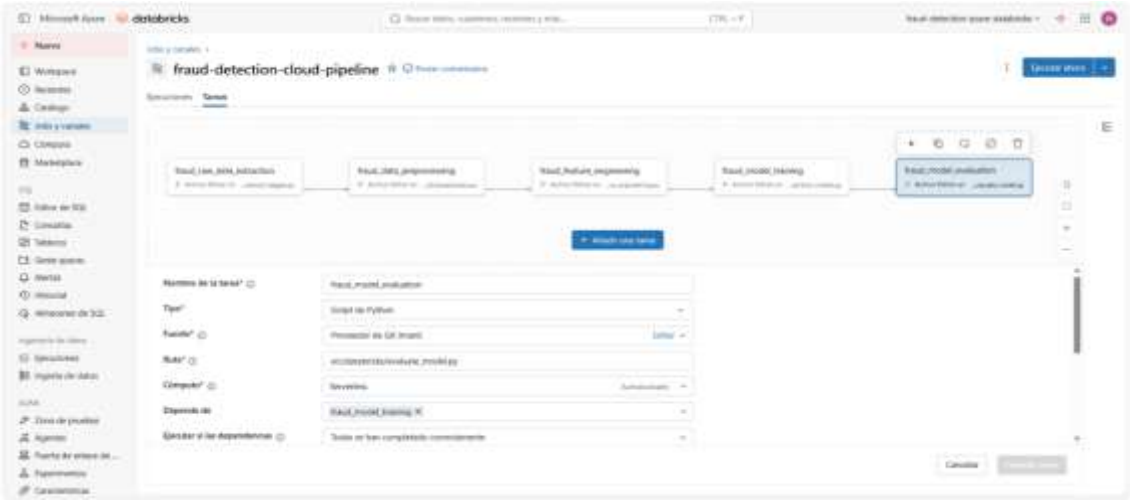

Figure 8. Azure Databricks Job for Cloud Pipeline Execution.

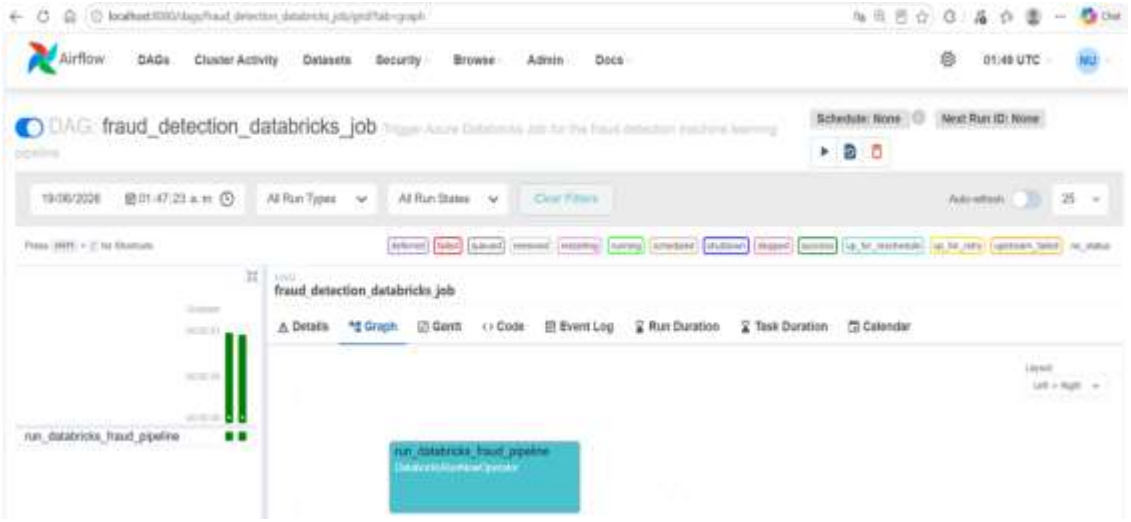

Figure 9. Cloud Pipeline Orchestration with Apache Airflow.